ALLIANCE SOFTWARE MANAGEMENT PLAN (SMP) TEMPLATE

NOTE:

This is the Alliance Software Management Plan (SMP) Template (version 1) consisting of 18 questions, including 13 mandatory questions (marked with *) and 5 optional questions.

Glossary

Software: Software in this SMP template refers to two categories: 1). software tools (e.g., software programs, languages, libraries, scripts, computational code, models, electronic lab notebooks, repository software, workflow management tools, etc.); and 2). software platforms (variously referred to as research infrastructures, virtual science labs, virtual research environments (VREs), or Science Gateways, etc.).

Software Versioning: In software development, software versioning is a type of numbering or naming scheme that helps software development teams keep track of changes they make to software project code by identifying successive versions. The changes may include new functions, features, or bug fixes. Occasionally, entirely new functions and features are released based on developments across multiple versions. As such, it assists in the creation and management of multiple software product releases. Modern computer software is often tracked using two different software versioning schemes: 1). an *internal* version number that may be incremented many times in a single day, such as a revision control number, and 2). a *release* version that typically changes far less often, such as semantic versioning or a project code name.

Version Control: Version control (also known as source control, revision control) is the practice of tracking and managing changes to software code over time. Version control is also a way to ensure efficient and collaborative code sharing and editing among multiple developers on different versions of the software at any given time within the larger system. Version control systems (VCS), sometimes known as SCM (Source Code Management) tools or RCS (Revision Control System), are software tools that help software teams manage changes to source code over time.

Software Release Life Cycle (SRLC): The Software release life cycle (SRLC) is a set of milestones that describe various stages in a piece of software's sequential release timeline, from its conception to its public distribution. It typically consists of several stages, such as pre-alpha (referring to the early stage of development where the software is still in its design and development phase), alpha (which represents the first formal testing phase using internal resources), beta (during which it's tested by a larger group of users outside of the organization to find and fix potential bugs or issues), release candidate (for further refinement and testing), and production release (also called stable release, marking the stable and complete version of the product ready for use by end-users). Once released, depending on the method of release, there are stages such as, release to manufacturing (RTM, also known as "going gold", when a software product is ready to be

delivered), general availability (GA, is a marketing stage when it becomes available for purchase), and release to the web (RTW, or web release, is a means of software delivery utilizing the internet for distribution).

Purpose

*** Provide a brief description of your software, stating its purpose, intended audience, and the problem it solves.**

The Codebook Generator is a web-based application developed using R and Shiny to assist researchers, data curators, and research support professionals in creating standardized, editable codebooks for tabular datasets. It allows users to upload datasets in .csv, .tsv, or .xlsx formats, preview their structure, and generate customizable metadata for each variable, including type, label, range or levels, units, and missing values. By simplifying and guiding the codebook creation process, this tool promotes consistent metadata practices and enhances the FAIRness (Findability, Accessibility, Interoperability, and Reusability) of research data. It addresses the common problem of incomplete or inconsistent variable documentation, which often hinders data sharing, reuse, and reproducibility. The app is deployable through ShinyLive, enabling full functionality in a browser without backend dependencies.

Documentation and Metadata

Please describe the architecture of the software (platforms). Are there some high-level principles that will be or were used as an inspiration for the design of the software (platforms)?

The architecture of the Codebook Generator App is based on the R programming language and leverages the Shiny framework to create an interactive web-based interface. Its structure follows a client-first design approach, enabled by the use of ShinyLive, which allows the entire application to run locally in the user's browser using WebAssembly. This design eliminates the need for a server backend, ensuring a privacy-respecting experience where no data leaves the user's machine. This is especially important for researchers working with sensitive or unpublished datasets, as it guarantees that data files remain completely under user control.

The interface is structured with a left-hand panel for input, and and a main panel on the right that renders previews of the uploaded data and an editable variable attributes table. This layout helps guide the user through a intuitive workflow. Internally, the app performs on-the-fly calculations of variable metadata, such as type, range or levels, and missing values. It also offers interactive fields for users to annotate each variable with labels and measurement units. These annotations are then compiled into a downloadable codebook in CSV format using browser-based JavaScript functions, avoiding any dependency on server-side resources.

*** Please describe how your documentation supports users and developers of your software (platforms). Provide links to existing documentation if available, including any documentation best practices you follow.**

The app is supported by openly accessible documentation through a dedicated webpage: https://alliance-rdm-gdr.github.io/RDM_CodebookGenerator/RDM_Codebook_en.html. The guide walks users through the full functionality of the application, from uploading datasets and completing metadata fields to downloading the final codebook. The documentation also explains key concepts such as variable labels, data types, value ranges, missing values, and units of measurement.

For developers and curators interested in the internal workings of the app, the documentation is complemented by clear source code that follows standard development practices. The code is organized into functional sections that separate the user interface from server logic, and it includes explanatory comments that describe the purpose of each component. The application is built using well-established R packages such as shiny, rhandsontable, and readxl.

*** How will you make sure that documentation is created or captured consistently throughout your project?**

To ensure that documentation is created and captured consistently, we use GitHub for version control and posting updates.

Testing and Deployment Strategy

Please describe or outline how you intend to test and deploy your software (platform) to ensure it: (1) meets the requirements that guided its design and development, (2) is usable and performs its intended function/s, and (3) can be installed and run in its intended environment.

The deployment strategy for the Codebook Generator is designed to ensure reliability, usability, and compatibility with its intended environment using web browsers via shinylive deployment. This method simplifies installation and guarantees that the software can be run in virtually any environment with a modern browser, such as Windows, macOS, and Linux systems. The app is hosted on GitHub Pages, making it freely accessible and easy to update as the code evolves.

The application was initially developed locally using the R shiny framework, and underwent iterative testing through manual validation, functional testing, and feedback from test users. Each feature was tested to confirm it behaves as intended under a range of conditions, including the handling of different file types (.csv, .tsv, and .xlsx), different data types (numeric, categorical, date). To verify that the application meets the initial design goals, early versions were shared with data curation professionals. This feedback directly informed the development of critical features such as data previewing, interactive table editing, automatic detection of value ranges and missing values.

Deployment is handled through shinylive, which allows the application to run entirely in the user's web browser without the need for an R server backend. Future enhancements may include incorporating automated testing using shinytest2 to validate the user interface and logic under multiple scenarios.

Version Control & Release Management

*** How do you plan to manage versioning for your software? What are specific functionalities provided by the version control system/platform?**

We manage source code with Git, hosting the repository on GitHub to take full advantage of version control, collaboration workflows, and releases. From day one, we adhere to Semantic Versioning, incrementing the major version for backwards-incompatible changes, and the minor version for backwards-compatible feature additions. Each time we prepare a public release, a Git tag corresponding to the new version (for example, v1.2.0). Within GitHub, we implement branch protection so all changes must arrive via pull requests. Every pull request is revised before merging. We also use GitHub Issues to track feature requests and bugs that will drive future version increments.

What strategies/high-level principles/mechanisms/practices/tools do you plan to utilize for your software release life cycle (SRLC)?

Our release life cycle is shaped by collaborative planning, where each new feature request or bug report is triaged in GitHub issues. Development occurs on short-lived feature branches, where contributors implement changes, update associated documentation, and write or extend tests. Continuous integration pipelines validate every commit through GUI tests, linting checks, and dependency audits.

Sharing and Reuse

*** Describe how you will make your software publicly available during and after development/upgrading, including through repositories like GitHub, Zenodo, or Figshare. Provide a rationale if open access is not feasible.**

The Codebook Generator will be made openly accessible via GitHub, where each commit, issue, and pull request is visible to the community; contributors can fork the project, submit changes, and track progress in real time. Upon tagging a new release, GitHub Actions will automatically package the application and publish a release entry containing executables, source archives, and release notes. To provide a citable, every major release on Zenodo, obtaining a DOI that can be referenced in scholarly publications and data management plans.

*** Specify the type of license for your software, preferably an open license. If not open, provide a justification. Consider hybrid licensing for software with multiple components.**

The Codebook Generator is distributed under the MIT License, an OSI-approved open-source license that grants users the freedom to use, copy, modify, merge, publish, distribute, sublicense, and sell the software without restriction, provided that the original copyright notice and license text are included in all copies or substantial portions of the software.

*** What steps will be taken to help the research community know that your software exists?**

To ensure that the research community becomes aware of the Codebook Generator, we will publish detailed announcements on the Digital Research Alliance of Canada's LinkedIn account, highlighting new capabilities and linking directly to the GitHub release notes. In parallel, we will submit the tool for inclusion in prominent software registries such as bio.tools, making it discoverable to those searching for "codebook generator" or "data dictionary generator" utilities. We also plan to send notification via our list-serv mailing list so all the RDM practitioners in Canada become aware of the tool.

*** How will users of your software be able to cite your software? Please provide a link to your software citation file (CFF) if available. (https://citation-file-format.github.io/)**

To facilitate proper attribution, Codebook Generator repository includes a machine-readable CITATION.cff file at the project root (https://github.com/Alliance-RDM-GDR/ RDM_CodebookGenerator/CITATION.cff). This file follows the Citation File Format standard and contains metadata such as the software title, version number, authors, DOI, and project URL. Users can incorporate the contents of CITATION.cff into their reference lists or use it with citation tools. For each major release archived on Zenodo, the DOI issued (e.g., 10.5281/zenodo.xxxxxxx) is also included in the citation metadata.

Preservation and Long-Term Maintenance

*** How and where will your software be archived, after the software is developed?**

After each major release, the Codebook Generator source code and executables will be secured on GitHub, with every commit, tag, and release retained indefinitely under the project's open-source license.

*** How do you plan to support long-term maintenance of your software?**

Ensuring the Codebook Generator remains reliable, Daniel Manrique-Castano, currently research data curator at the Digital Research Alliance of Canada will take the responsibility of monitoring issue queues, triaging bug reports, and reviewing community pull requests. Second, clear contribution guidelines and a well-documented codebase lower the barrier for new volunteer contributors; we actively encourage community members to take ownership.

If using an existing software component/platform, how would you deal with upgrades / patches to third-party software packages that you might use?

The Codebook Generator application is built using the R programming language and leverages several third-party packages, including shiny, rhandsontable, readxl, DT, shinythemes, and shinyBS. These components form the foundation for the user interface, data processing, and table editing. Because these libraries are actively maintained by the R community, it is expected that updates, patches, and new versions will be released periodically. To ensure the long-term stability and compatibility of the Codebook Generator, the development environment and package versions are carefully documented. For example, a DESCRIPTION or renv.lock file could be used to snapshot the exact package versions used during development and testing.

When updates to third-party packages become available, the strategy is to test them in a controlled development environment before applying them to the production version of the app. Minor patches (e.g., bug fixes or security updates) are typically integrated immediately after confirming compatibility. In contrast, major upgrades are first evaluated for backward compatibility, especially with core dependencies like shiny and rhandsontable. Additionally, GitHub is used as the source control platform, which allows tracking of changes and easy rollback in case an update causes unexpected behavior

User Support

*** After the software/research software platform has been developed, please include plans for any support-related activities (e.g., platform operations, providing user support, extending / adding functionality, facilitating adoption by new research teams, etc.) in terms of training, support staff allocation, communication channels.**

Software webpage	https://github.com/Alliance-RDM-GDR/RDM_Codebook_App
Software documentation	https://github.com/Alliance-RDM-GDR/RDM_Codebook_App/README.md
Software tutorials	Alliance YouTube channel

Responsibilities and Resources

What resources will you require to implement your software management plan? What do you estimate the overall cost for software management to be?

Implementing and sustaining the Software Management Plan for the File Tree Generator requires light human and technical resources. We anticipate building new features, maintain dependencies, correct bugs and issues. Being open-source in GitHub facilitates the task by the curation team or other RDM practitioners.

*** How will responsibilities for managing software activities be handled if substantive changes happen in the personnel overseeing the project's software, including a change of Principal Investigator? Please consider the situation for managing software both during and after the project.**

In the event that personnel change, we established a succession process to ensure uninterrupted stewardship of the Codebook Generator. All roles and responsibilities are recorded in our CONTRIBUTING.md, including primary maintainers and at least two secondary contacts who have full repository and release-management privileges.

Other Concerns

*** Describe the main external factors that should be considered by developers and users of the software. These could include any security-related information or concerns.**

Developers and users of the Codebook Generator App should consider several external factors that may influence the performance and and responsible use of the software. First and foremost, data privacy and security are central concerns. Although the app is intentionally designed not to store or transmit any uploaded data, clarified by a message within the interface, the user must still ensure that sensitive or personal data is not inadvertently exposed when using the app in shared or public computing environments. Since the application is deployed using ShinyLive and runs entirely within the user's browser, no data is uploaded to external servers. However, users must still exercise caution when opening datasets containing identifiable information.

Another important external factor is browser compatibility. As the application relies on JavaScript-based rendering (e.g., for rHandsontable and dynamic file downloads), performance and display may vary slightly depending on the browser and device used. The app is tested primarily on modern browsers such as Chrome, and developers should regularly verify that the responsive layout, scroll behavior, and embedded scripts continue to function properly across platforms and screen sizes. Developers should also be mindful of evolving web standards and ShinyLive updates, which may introduce breaking changes or deprecate functions relied upon by the app.

Accessibility standards are another relevant external factor. The app already incorporates a number of features to improve accessibility, such as keyboard navigability, readable font sizes, and high-contrast text for visually impaired users, but full WCAG compliance should be periodically reviewed. Developers should also monitor the maintenance and development of upstream packages like shiny, and rhandsontable, which play critical roles in the app’s functionality.

Date and Sign

The authors of this document will ensure that this Software Management Plan is carried out as specified above.

Name: Daniel Manrique-Castano

Affiliation: Digital Research Alliance of Canada

Date: 2025-07-23

Signature: